

FIELDPRO | PROOF OF CONCEPT

FieldPro POC and MAUI Learning Architecture

Implementation mapping for XAML, MVVM, mobile architecture, and platform integration.

Prepared 10 September 2026

Scope: Phase 1 UI foundation, Phase 2 mobile architecture, and Phase 3 platform integration.

Status: current repository implementation mapped against the training objectives.

1. Executive Summary

FieldPro is a cross-platform .NET MAUI field-service application. The current POC provides a login experience, a dashboard, work-order data, and logout behavior. Authentication is implemented locally for the POC using Entity Framework Core with SQLite and secure device storage.

Implemented • MAUI UI with Shell navigation. • Login and logout workflow. • EF Core SQLite user store. • PBKDF2 password hashing with per-user salt. • SecureStorage-backed authenticated user ID. • SQLite work-order persistence and seed data.	POC Boundaries • Authentication is device-local, not server-backed. • One development user is seeded from JSON. • No registration, password reset, roles, refresh tokens, or remote API yet. • Work orders currently use the existing sqlite-net data layer.
--	---

2. POC Goals

Goal	Result
Prove a real login flow	Credentials are verified against an EF Core user record instead of a hard-coded conditional.
Keep passwords out of the database in plain text	Passwords are transformed with PBKDF2-SHA256, 100,000 iterations, a random 16-byte salt, and a 32-byte derived key.
Keep the session across app navigation	The authenticated user ID is stored in platform-backed MAUI SecureStorage.
Provide reliable logout	Logout removes the secure user ID and clears the in-memory application session.

3. Phase Implementation Map

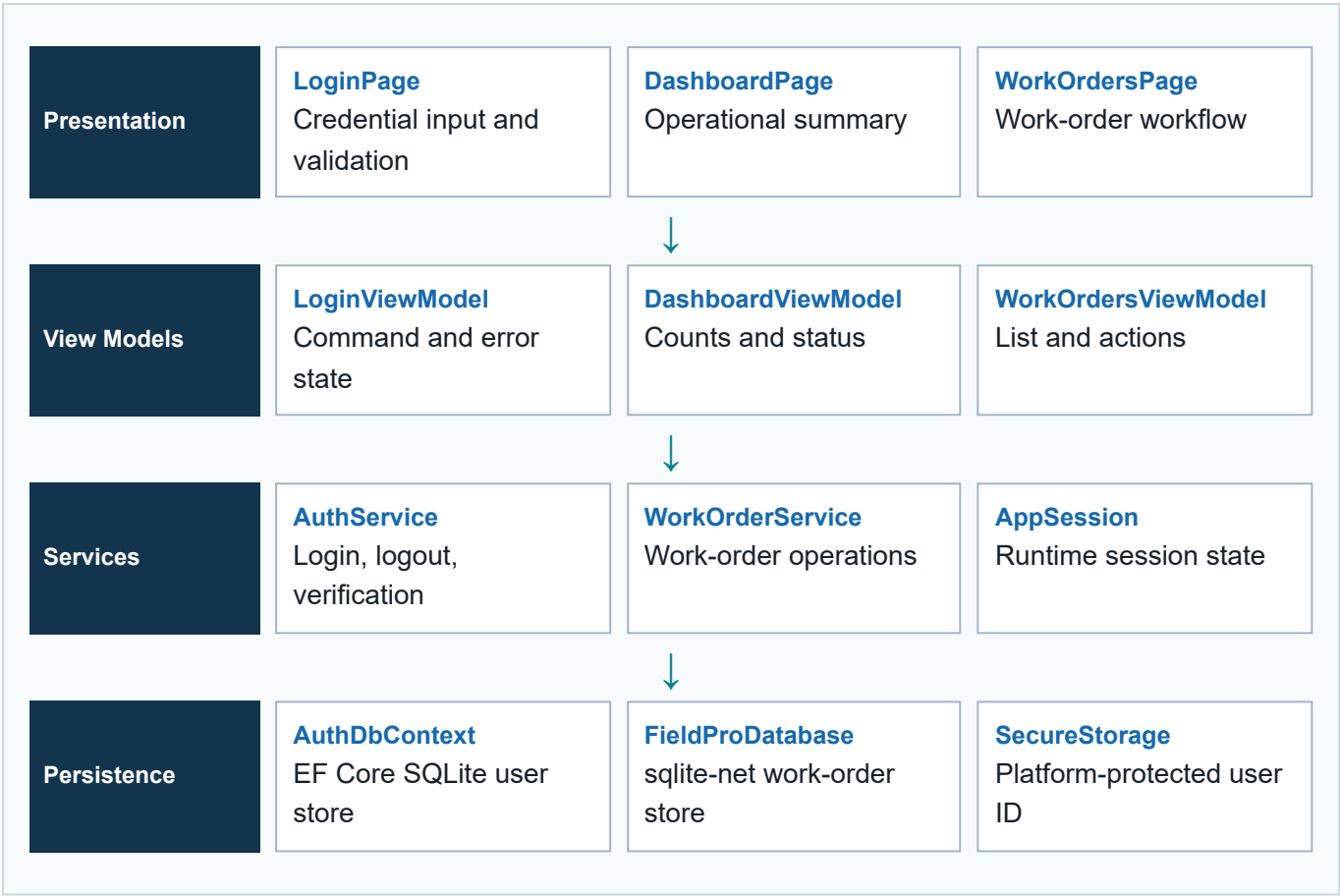
This matrix maps the requested learning objectives to the code currently present in FieldPro. Status is based on observable implementation in the repository.

Phase	Capability	Status	Current implementation
Phase 1 UI Foundation	XAML fundamentals	Implemented	Pages use XAML namespaces, controls, bindings, and nested visual-element hierarchies.
	Layouts and controls	Partial	Grid , VerticalStackLayout , ScrollView , Border , Entry , Button , and ActivityIndicator are used. FlexLayout and AbsoluteLayout are not currently used.
	MVVM separation	Implemented	Login, dashboard, work-order list, and work-order details have dedicated ViewModels and injected services.
	Data binding	Implemented	XAML binds to ViewModel properties such as Email , Password , counts, status, busy state, and error visibility.
	Commands	Implemented	CommunityToolkit [RelayCommand] generates commands for login, work-order actions, refresh, and camera completion.
	Property change notification	Implemented	CommunityToolkit [ObservableProperty] generates observable properties and notification code.
Phase 2 Mobile Architecture	Shell navigation	Implemented	AppShell defines Login and Main routes; work-order details use Shell route navigation and QueryProperty .
	Lifecycle and state	Partial	Pages use OnAppearing for loading and UI state. Explicit OnStart , OnSleep , and OnResume handling is not yet implemented.
	Dependency injection	Implemented	MauiProgram registers databases, services, ViewModels, and pages using Microsoft.Extensions.DependencyInjection .
	Local storage	Implemented	sqlite-net stores work orders, EF Core SQLite stores users, FileSystem stores captured photos, and SecureStorage stores the local user ID.
	Offline-first behavior	Partial	Local persistence exists, but synchronization, conflict resolution, and an offline queue are not yet implemented.
Phase 3 Platform Integration	Device capabilities	Partial	MediaPicker captures completion photos and FileSystem stores them. GPS and accelerometer integration are not present.
	Permissions	Gap	Camera availability is checked, but explicit runtime permission request and denial handling are not yet implemented.
	Platform-specific customization	Gap	No custom handlers, mappers, or platform compiler directives are currently used in the application code.

	Responsive UI	Partial	Grid and adaptive MAUI layouts provide a base; OnPlatform, OnIdiom, and dedicated phone/tablet/desktop resources are not yet used.
--	---------------	---------	--

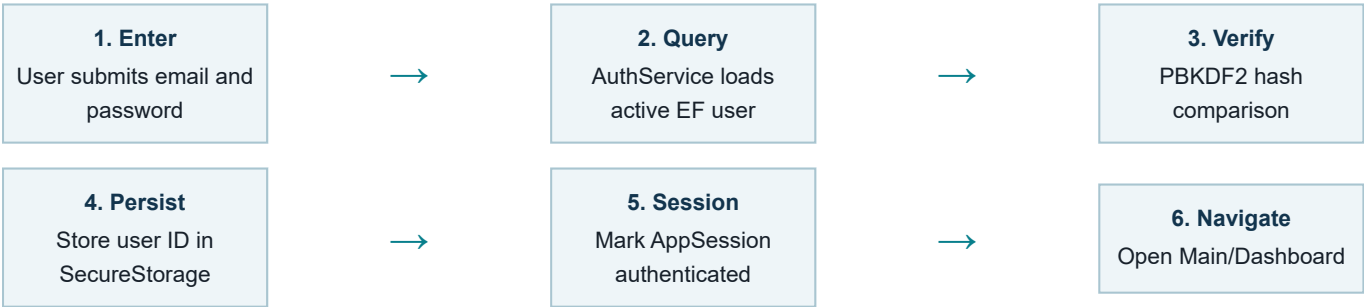
Training conclusion: Phase 1 is the strongest current implementation. Phase 2 is substantially represented through Shell, DI, and local persistence. Phase 3 is started with camera capture but needs explicit permissions and platform-specific work to be complete.

4. High-Level Architecture



Dependency injection is configured in **MauiProgram**. Shell controls route access and the logout menu action. Database initialization runs during startup on a background task.

Authentication Request Flow



Logout Flow

Shell resolves **IAuthService**, removes the SecureStorage user ID, clears **IAppSession**, and navigates to the login route with a cleared stack.

5. Data and Security Design

Auth database EF Core uses a dedicated SQLite file named fieldpro-auth.db3 . The User entity contains an ID, unique email, password hash, password salt, and active flag.	Work-order database The existing sqlite-net database remains separate from authentication. This limits schema coupling while the POC is being developed.
---	--

Concern	Current POC treatment	Production evolution
Credential seed	auth-seed.json is packaged for initial local setup only.	Remove default credentials from the client and provision accounts through an identity service.
Password storage	Only derived hash and salt are stored in EF.	Use a managed identity provider or a server-side password policy and audit trail.
Session token	Only a local user ID is stored in SecureStorage.	Use short-lived access tokens and rotating refresh tokens from an API.
Authorization	Active-user check only.	Add roles, permissions, tenant boundaries, and server-side authorization.
Transport	No remote transport in this POC.	Use HTTPS, certificate validation, API request timeouts, and error handling.

Important: The credentials in **auth-seed.json** are development credentials. They must not be shipped unchanged in a production release.

6. POC Acceptance Criteria

- First launch creates the auth SQLite database and seeds a user from JSON when no users exist.
- Valid credentials navigate to the dashboard.
- Invalid credentials do not create a session and show an error.
- Restarting the app preserves a valid local session token.
- Logout removes the persisted session and returns to login.
- Windows target builds successfully.

7. Recommended Next Steps

1. **Complete Phase 2 lifecycle handling:** add centralized app lifecycle state restoration and resource cleanup for backgrounding and resume.
2. **Complete Phase 3 permissions:** request camera permission explicitly and handle denied, restricted, and permanently denied states.
3. **Add responsive resources:** use OnIdiom and OnPlatform styles for phone, tablet, desktop, Android, iOS, and Windows differences.
4. **Add platform customization:** introduce a small handler mapper example only where the native control behavior requires it.
5. **Introduce an API boundary:** move credential verification and user management to a backend service.
6. **Replace the seed account:** remove the packaged password and add controlled provisioning for development environments.
7. **Unify persistence:** migrate work orders from sqlite-net to EF Core migrations if a single relational model is preferred.
8. **Add authorization:** model technician, supervisor, and administrator permissions.
9. **Add automated tests:** cover hash verification, invalid credentials, logout, inactive users, and database initialization.
10. **Add observability:** structured authentication events without logging passwords or tokens.
11. **Plan offline sync:** define conflict handling, local queueing, and server reconciliation for field connectivity.

8. Implementation Reference

Area	Primary files
Authentication	FieldPro/Services/AuthService.cs FieldPro/Services/IAuthService.cs
EF Core model	FieldPro/Data/AuthDbContext.cs FieldPro/Models/User.cs
Seed configuration	FieldPro/Resources/Raw/auth-seed.json
Startup and DI	FieldPro/MauiProgram.cs
Navigation and logout	FieldPro/AppShell.xaml.cs